Experiment 12.1.1: Phone Number Database Management

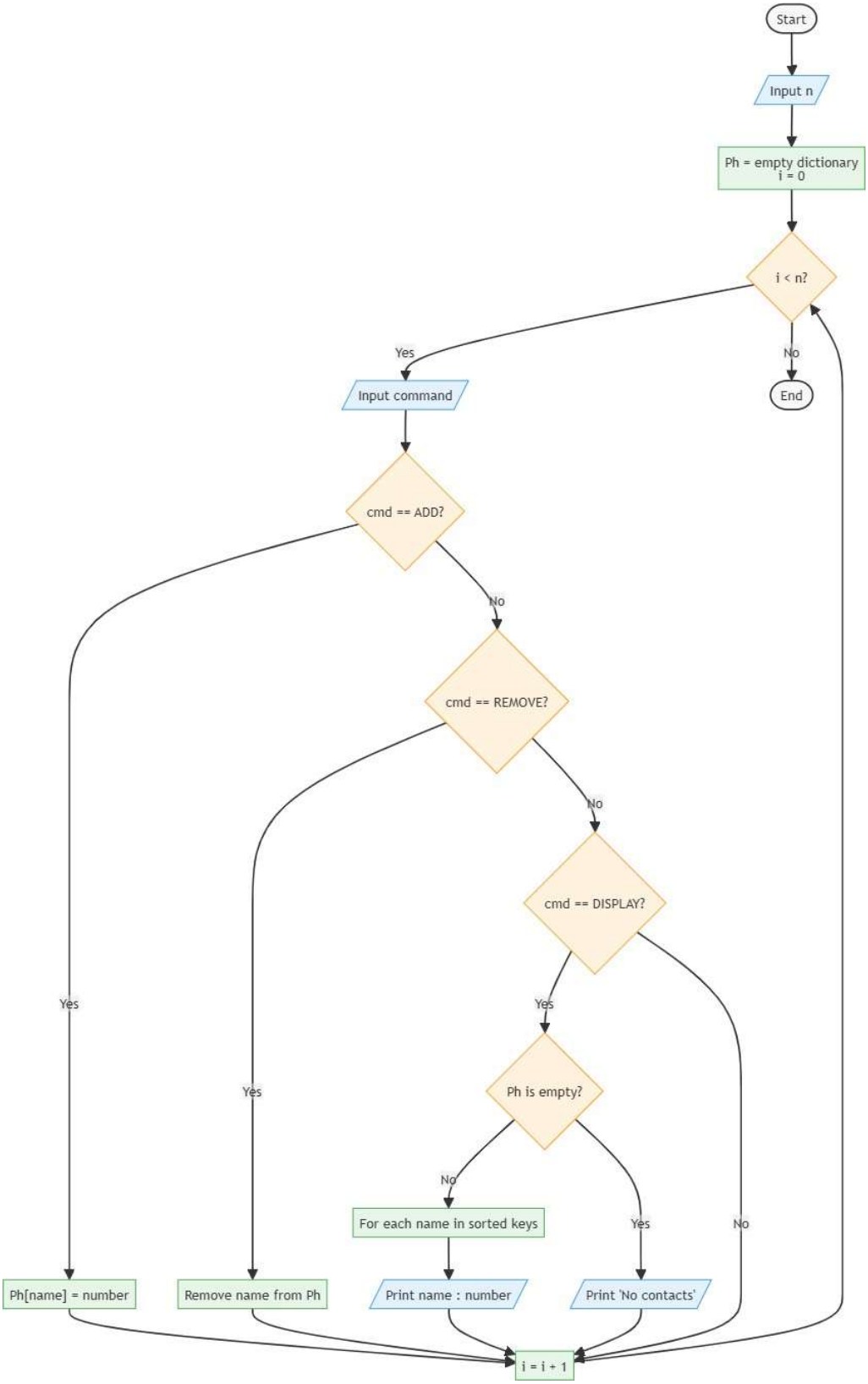
1. Aim

To design and implement a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values. The program supports three operations: adding a contact (or updating if it exists), removing a contact (ignoring if it doesn't exist), and displaying all contacts in alphabetical order by name.

2. Pseudocode

1. **START**
2. **READ** an integer n representing the total number of operations to perform.
3. **INITIALIZE** an empty dictionary named Ph.
4. **FOR** i from 1 to n:
    - **READ** the operation command and parameters as a list cmd.
    - **IF** cmd[0] is "ADD":
        - Add or update the contact: Ph[cmd[1]] = cmd[2].
    - **ELSE IF** cmd[0] is "REMOVE":
        - Remove the contact if it exists using Ph.pop(cmd[1], None).
    - **ELSE IF** cmd[0] is "DISPLAY":
        - **IF** the dictionary Ph is empty:
            - **PRINT** "No contacts".
        - **ELSE**:
            - **SORT** the dictionary keys (names) alphabetically.
            - **FOR** each name in the sorted keys:
                - **PRINT** the contact in the format {name}: {phone_number}.
5. **END**

3. Flowchart


Start
Input n
Ph = empty dictionary
i = 0
i < n?
Yes
No
End
Input command
cmd == ADD?
No
cmd == REMOVE?
No
cmd == DISPLAY?
Yes
Yes
Yes
Ph is empty?
No
For each name in sorted keys
Yes
No
Ph[name] = number
Remove name from Ph
Print name : number
Print 'No contacts'
i = i + 1

4. Experiment Screenshot

CODETANTRA Home

shreyansh.shirbhate.batch2025@sitnagpur.siu.edu.in Support Logout

12.1.1. Phone Number Database Management 00:16

Write a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values.

The program should support three operations: adding a contact (if the name already exists, update the phone number), removing a contact (if the name does not exist, ignore the operation), and displaying all contacts in alphabetical order by name.

Input Format:

1. First line contains an integer n representing the number of operations.
2. Each of the next n lines contains a single operation in one of the following formats:
   - "ADD <name> <phone_number>" where <name> is a string, <phone_number> is a string of 10 digits.
   - "REMOVE <name>" - name is a string.
   - "DISPLAY" - no additional parameters.

Output Format:

- For display operation, print all contacts in alphabetical order by name in the format:

```
<name>: <phone_number>
```

- If the database is empty during display operation, print: "No contacts".
- Do not print anything for add and remove operations.

Constraints:

- Name contains only alphabetic characters (no spaces). And it is case sensitive.

Sample Test Cases +

Explorer

phoneDat...

Submit

```
n = int(input())
contacts = {}
for _ in range(n):
    parts = input().split()
    if parts[0] == "ADD":
        name = parts[1]
        number = parts[2]
        contacts[name] = number
    elif parts[0] == "REMOVE":
        name = parts[1]
        if name in contacts:
            del contacts[name]
    elif parts[0] == "DISPLAY":
        if not contacts:
            print("No contacts")
    else:
```

Debugger

Average time	Maximum time
0.007 s	0.009 s
6.88 ms	9.00 ms

5 out of 5 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 8 ms Debug

Expected output	Actual output
4	4
ADD Alice 9876543210	ADD Alice 9876543210
ADD Bob 8765432109	ADD Bob 8765432109
ADD Charlie 7777777777	ADD Charlie 7777777777
DISPLAY	DISPLAY
Alice: 9876543210	Alice: 9876543210

Terminal Test cases